

Description Logics and the Semantic Web

Umberto Straccia

ISTI - CNR,

<http://gaia.isti.cnr.it/~straccia>

straccia@isti.cnr.it

2006

Outline

1 The Semantic Web and Ontologies

- Today's Web
- The Semantic Web Impact
- Semantic Web Technologies
 - Explicit Metadata
 - Ontologies
 - Web Ontology Languages
 - RDF and RDFS
 - OWL

2 From OWL to Description Logics

- What Are Description Logics?
- Short History of Description Logics
- DLs Basics
- OWL DL and OWL Lite as Description Logic

3 Description Logic Reasoning

- Basic Inference Problems (Formally)
- Reasoning in DLs: Basics
 - $\mathcal{ALC}$ Tableau rules

Foundations of the Semantic Web

Ontologies and OWL

<http://www.semanticweb.org/>

Today's Web

- Most of today's Web content is suitable for human consumption
 - ▶ even Web content that is generated automatically from databases is usually presented without the original structural information found in databases
- Typical Web uses today people's
 - ▶ seeking and making use of information, searching for and getting in touch with other people, reviewing catalogs of online stores and ordering products by filling out forms

Keyword-Based Search Engines

- Current Web activities are not particularly well supported by software tools
 - ▶ Except for keyword-based search engines (e.g. Google, AltaVista, Yahoo)
- The Web would not have been the huge success it was, were it not for search engines

Problems with Keyword-based Search Engines

- High recall, low precision
- Low or no recall
- Results are highly sensitive to vocabulary
- Results are single Web pages
- Human involvement is necessary to interpret and combine results
- Results of Web searches are not readily accessible by other software tools

Impossible (?) Using Today's Web

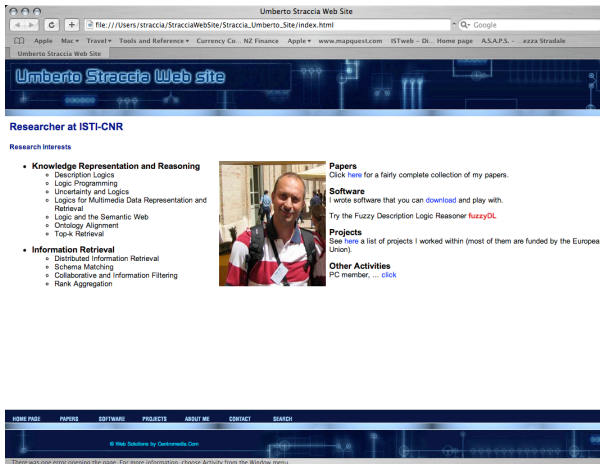
- Complex queries involving background knowledge
 - ▶ Find information about “animals that use sonar but are not either bats or dolphins”, e.g. Barn Owl



- Book me a holiday next weekend somewhere warm, not too far away, and where they speak French or English

What is the Problem?

- Consider a typical web page. Markup consists of:
 - rendering information (e.g., font size and colour)
 - Hyper-links to related content



- Semantic content is accessible to humans but not (easily) to computer ...

The Key Problem of Today's Web

- The meaning of Web content is not machine-accessible
 - ▶ lack of semantics
- It is simply difficult to distinguish the meaning between these two sentences:
 - ▶ “I am a soccer player”
 - ▶ “You may think I’m a soccer player”

The Semantic Web Approach

- Represent Web content in a form that is more easily machine-processable
- Use intelligent techniques to take advantage of these representations
- The Semantic Web will gradually evolve out of the existing Web, it is not a competition to the current WWW

The Semantic Web Impact - Knowledge Management

- Knowledge management concerns itself with acquiring, accessing, and maintaining knowledge within an organization
- Key activity of large businesses: internal knowledge as an intellectual asset
- It is particularly important for international, geographically dispersed organizations
- Most information is currently available in a weakly structured form (e.g. text, audio, video)

Limitations of Current Knowledge Management Technologies

- Searching information
 - ▶ Keyword-based search engines
- Extracting information
 - ▶ Human involvement necessary for browsing, retrieving, interpreting, combining
- Maintaining information
 - ▶ Inconsistencies in terminology, outdated information
- Viewing information
 - ▶ Impossible to define views on Web knowledge

Semantic Web Enabled Knowledge Management

- Knowledge will be organized in conceptual spaces according to its meaning
- Automated tools for maintenance and knowledge discovery
- Semantic query answering
- Query answering over several documents
- Defining who may view certain parts of information (even parts of documents) will be possible

The Semantic Web Impact - B2C Electronic Commerce

- A typical scenario: user visits one or several online shops, browses their offers, selects and orders products
- Ideally humans would visit all, or all major online stores; but too time consuming
- **Shopbots** are a useful tool

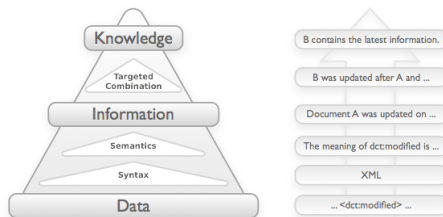
Limitations of Shopbots

- They rely on wrappers: extensive programming required
- Wrappers need to be reprogrammed when an online store changes its outfit
- Wrappers extract information based on textual analysis
 - ▶ Error-prone
 - ▶ Limited information extracted

Semantic Web Enabled B2C Electronic Commerce

- Software agents that can interpret the product information and the terms of service
 - ▶ Pricing and product information, delivery and privacy policies will be interpreted and compared to the user requirements
- Information about the reputation of shops
- Sophisticated shopping agents will be able to conduct automated negotiations

Summary of knowledge flow process



- Data = Documents, Pages, Blobs...
- Structured Data = Data + Syntax
- Semantics = Meaning of Structured Data elements
- Information = Structured Data + Semantics
- Knowledge = Purposeful Combination of Information
- The Web: A Data repository
- The Semantic Web = Putting Semantics on the Web

Semantic Web Technologies

- Explicit Metadata
- Ontologies
- Logic and Inference

On HTML

- Web content is currently formatted for human readers rather than programs
- HTML is the predominant language in which Web pages are written (directly or using tools)
- Vocabulary describes presentation

An HTML Example

```
html.html
Last Saved: 21-11-2006 18:59:27
File Path: ~/Research/Didattica/Course/UNIP/html.html

<html>
<head>
  <meta http-equiv="content-type" content="text/html; charset=ISO-8859-1">
  <title>Homepage</title>
  <link href="css.css" rel="stylesheet" media="screen">
  <meta name="description" content="">
  <meta name="keywords" content="">
  <meta name="generator" content="www.CentroMedia.Co">
  <meta name="copyright" content="www.CentroMedia.Co" by Giulio Straccia, Italy. All contents Copyright. All rights reserved. Reproduction in whole or in part
  <meta name="author" content="www.CentroMedia.Co">
  <script src="noses" SRC="BSPBC95"></script>
  <script>
    <script name="BSPBC95" class="Netscape CSS Fix" type="text/javascript"></script>
  </script>
  <script type="text/javascript" src="Generated/tees/CSScriptLib.js"></script>
  </script>
  <script type="text/javascript">
    CSRef["CSP"/"BSPBC95"] = new Array(CSPFix);
    CSRef["CSP"/"CSP"] = new Array(CSPFix);
  </script>
  </head>
  <body>
    <div align="left">
      <p class="titolo">Researcher at ISTI-CNR</p>
      <p class="titolo"><font size="2">Research Interests</font></p>
    </div>
    <table border="0" cellpadding="0" cellspacing="0">
      <tr valign="top">
        <td valign="top">
          <ul>
            <li class="testo"><span class="sottitolo">Knowledge Representation and Reasoning </span>
            <ul>
              <li class="testo">Description Logics
              <li class="testo">Logic Programming
              <li class="testo">Uncertainty and Logic
              <li class="testo">Logics for Multimedia Data Representation and Retrieval
            </ul>
            <li class="testo">Logic and the Semantic Web
            <li class="testo">Ontology Alignment
            <li class="testo">Top-k Retrieval
          </ul>
          <ul>
            <li class="testo"><span class="sottitolo">Information Retrieval</span>
            <ul>
              <li class="testo">Distributed Information Retrieval
              <li class="testo">Schema Matching
              <li class="testo">Collaborative and Information Filtering
              <li class="testo">Rank Aggregation
            </ul>
          </ul>
        </td>
        <td valign="top" width="100"> <div align="left">
          <p class="testo"><span class="sottitolo">Publications</span></div>
          <p class="testo"><span class="sottitolo">Click on <a href="papers/index.html" title="Ubaldo Straccia's publication list">here</a> for a fairly complete collection of my papers.</p>
          <p class="testo"><span class="sottitolo">I wrote software that you can <a href="software/index.html" title="Ubaldo Straccia's developed software list">download</a> and play with.</p>
          <p class="testo"><span class="sottitolo">Try the Fuzzy Description Logic Reasoner <a href="software/fuzzyDL/fuzzyDL.html">here</a> for a fairly complete collection of my papers.</p>
          <p class="testo"><span class="sottitolo">See <a href="projects.html" title="Ubaldo Straccia's projects list">here</a> for a list of projects I worked within (most of them are funded by
          <p class="testo"><span class="sottitolo">PC member <a href="others.html" title="Other activities">here</a></p>
        </td>
      </tr>
    </table>
    <div align="left">
      <p class="testo"><span class="sottitolo">Shingstat Free code <a href="http://codice.shingstat.com/cgi-bin/getcod.cgi?USER=traccia">here</a></p>
    </div>
  </body>
</html>
```

Problems with HTML

- Humans have no problem with this
- Machines (software agents) do:
 - ▶ How distinguish my research topics from my hobbies
 - ▶ How determine my location

Better Representation

```
<topicList>
  <category>
    <name> Knowledge Representation and Reasoning </name>
    <topic> Description Logic </topic>
    <topic> Logic Programming </topic>
    <topic> Uncertainty and Logics </topic>
  </category>
  <category>
    <name> Information Retrieval </name>
    <topic> Distributed Information Retrieval </topic>
    <topic> Schema Matching </topic>
    <topic> Rank Aggregation </topic>
  </category>
</topicList>
```

Explicit Metadata

- This representation is far more easily processable by machines
- Metadata: data about data
 - ▶ Metadata capture part of the meaning of data
- Semantic Web does not rely on text-based manipulation, but rather on machine-processable metadata

Ontologies

- The term ontology originates from philosophy
 - ▶ The study of the nature of existence
- Different meaning from computer science
 - ▶ An ontology is an explicit and formal specification of a conceptualization
 - ▶ Formal and machine manipulable model of a domain of interest

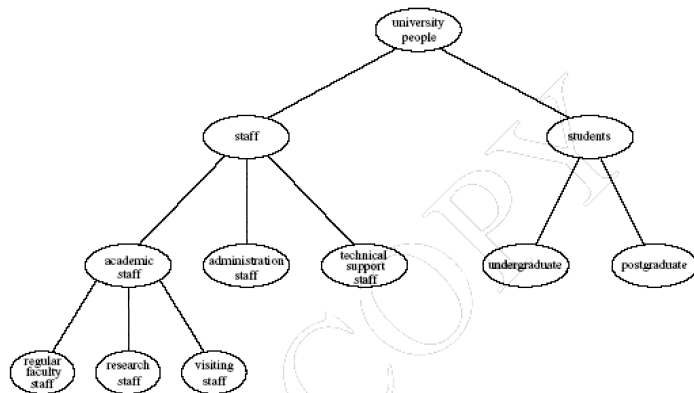
Typical Components of Ontologies

- **Terms** denote important concepts (classes of objects) of the domain
 - ▶ e.g. professors, staff, students, courses, departments
 - ▶ New concepts can be formed by combining existing ones (e.g. a male professor)
 - ▶ Meaning (semantics) of such terms is formally specified
- **Relationships** between these terms: typically class hierarchies
 - ▶ a class C to be a subclass of another class C' if every object in C is also included in C'
 - ▶ e.g. all professors are staff members

Further Components of Ontologies

- Properties:
 - ▶ e.g. X teaches Y
- Value restrictions
 - ▶ e.g. only faculty members can teach courses
- Disjointness statements
 - ▶ e.g. faculty and general staff are disjoint
- Logical relationships between objects
 - ▶ e.g. every department must include at least 10 faculty

Example of a Class Hierarchy

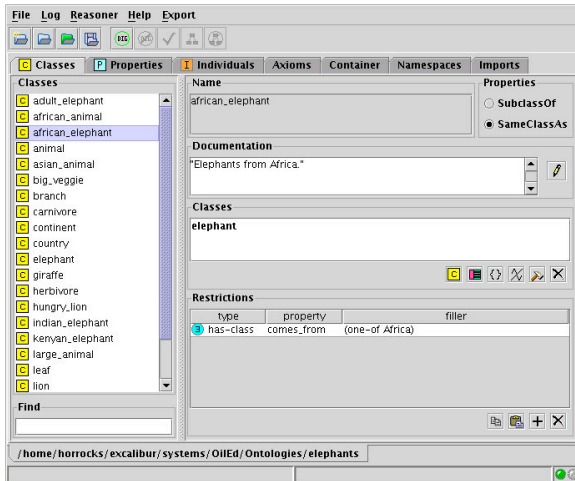


Another Example of Structure of an Ontology

Ontologies typically have two distinct components:

- Names for important concepts in the domain
 - ▶ **Elephant** is a concept whose members are a kind of animal
 - ▶ **Herbivore** is a concept whose members are exactly those animals who eat only plants or parts of plants
 - ▶ **Adult_Elephant** is a concept whose members are exactly those elephants whose age is greater than 20 years
- Background knowledge/constraints on the domain
 - ▶ **Adult_Elephants** weigh at least 2,000 kg
 - ▶ All **Elephants** are either **African_Elephants** or **Indian_Elephants**
 - ▶ No individual can be both a **Herbivore** and a **Carnivore**





<http://protege.stanford.edu/>

A Semantic Web – First Steps

Make web resources more accessible to automated processes

- Extend existing rendering markup with **semantic markup**
 - ▶ Metadata annotations that describe content/function of web accessible resources
- Use Ontologies to provide **vocabulary** for annotations
 - ▶ “Formal specification” is accessible to machines
- A prerequisite is a standard web ontology language
 - ▶ Need to agree common **syntax** before we can share semantics
 - ▶ Syntactic web based on **standards** such as **HTTP** and **HTML**

The Role of Ontologies on the Web

- Ontologies provide a shared understanding of a domain: **semantic interoperability**
 - ▶ overcome differences in terminology
 - ▶ mappings between ontologies
- Ontologies are useful for the organization and navigation of Web sites

The Role of Ontologies in Web Search

- Ontologies are useful for improving the accuracy of Web searches
 - ▶ search engines can look for pages that refer to a precise concept in an ontology
- Web searches can exploit generalization/ specialization information
 - ▶ If a query fails to find any relevant documents, the search engine may suggest to the user a more general query
 - ▶ If too many answers are retrieved, the search engine may suggest to the user some specializations

Ontology Design and Deployment

Given key role of ontologies in the Semantic Web, it will be essential to provide **tools** and **services** to help users:

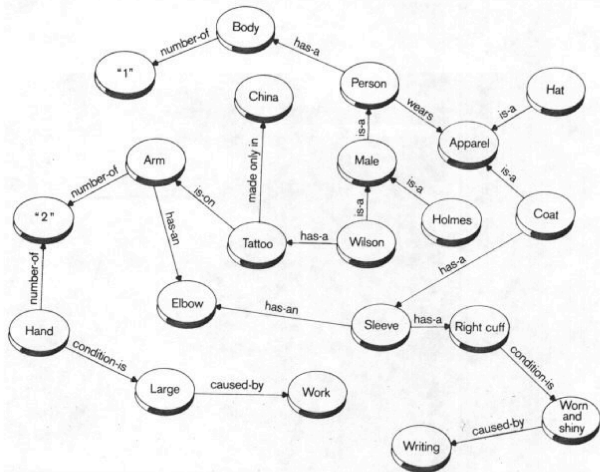
- Design and maintain high quality ontologies, e.g.:
 - ▶ **Meaningful** – all named classes can have instances
 - ▶ **Correct** – captured intuitions of domain experts
 - ▶ **Minimally redundant** – no unintended synonyms
 - ▶ **Richly axiomatised** – (sufficiently) detailed descriptions
- Store (large numbers) of **instances** of ontology classes, e.g.:
 - ▶ Annotations from web pages
- Answer **queries** over ontology classes and instances, e.g.:
 - ▶ Find more general/specific classes
 - ▶ Retrieve annotations/pages matching a given description
- **Integrate** and align multiple ontologies

Web Ontology Languages

- Wide variety of languages for “Explicit Specification”

- ▶ **Graphical notations**

- ★ Semantic networks

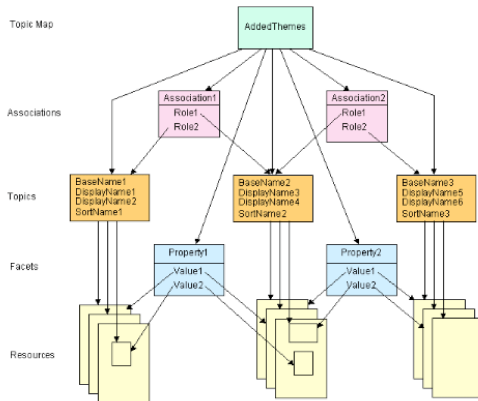


Web Ontology Languages

- Wide variety of languages for “Explicit Specification”

- ▶ Graphical notations

- ★ Topic Maps (see <http://www.topicmaps.org/>)

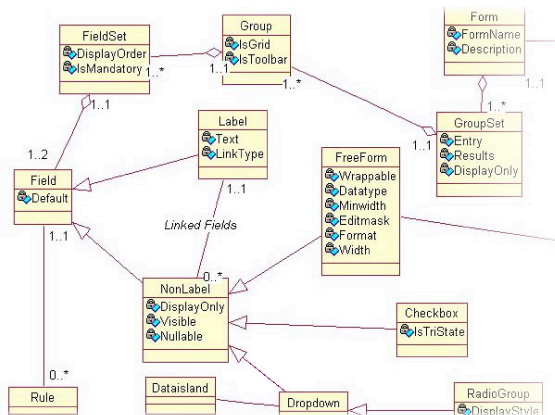


Web Ontology Languages

- Wide variety of languages for “Explicit Specification”

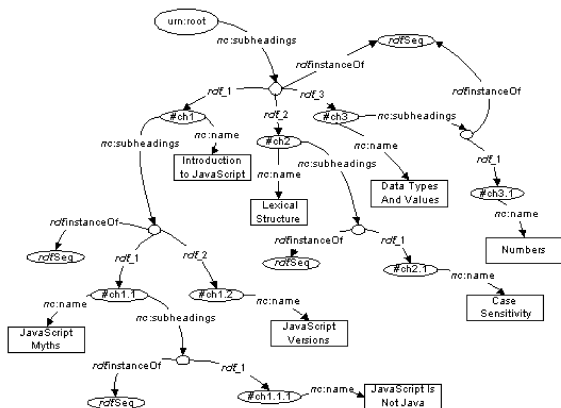
- ▶ Graphical notations

- ★ UML



Web Ontology Languages

- Wide variety of languages for “Explicit Specification”
 - ▶ Graphical notations
 - ★ RDF



- Wide variety of languages for “Explicit Specification”

- ▶ Logic based

- ★ Description Logics (e.g., OIL, DAML+OIL, OWL)

“A Happy father is a Man, which has a daughter”

Happy_Father $\sqsubseteq$ *Man* $\sqcap \exists has_child.Female$

- Wide variety of languages for “Explicit Specification”

- ▶ Logic based

- ★ Rules (e.g., RuleML, LP/Prolog)

“A Man, which has a daughter is a Happy father”

Happy_Father(x) ← Man(x), Has_child(x, y), Female(y)

- Wide variety of languages for “Explicit Specification”

- ▶ Logic based

- ★ First Order Logic (e.g., KIF)

“A Man, which has a daughter buys a lot of clothes”

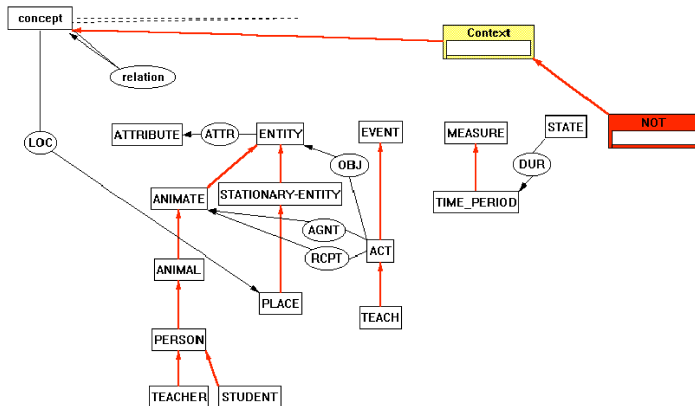
$\forall x. Man(x) \wedge Has_child(x, y) \wedge Female(y) \rightarrow BuysALot(x, y) \wedge Clothes(y)$

Web Ontology Languages

- Wide variety of languages for “Explicit Specification”

- ▶ Logic based

- ★ Conceptual graphs



Web Ontology Languages

- Wide variety of languages for “Explicit Specification”
 - ▶ Logic based
 - ★ Non-classical logics (e.g., Flogic, Non-Monotonic, Modal Logics)

Web Ontology Languages

- Wide variety of languages for “Explicit Specification”

- ▶ Probabilistic/Fuzzy

80% of Birds fly

$\langle Bird(x) \rightarrow Fly(x), 0.8 \rangle$

A sports car is very fast

$\langle \forall x. SportsCar(x) \wedge HasSpeed(x, y) \wedge very(High(y)) \rangle$

Web Ontology Languages: Summary

- Wide variety of languages for “Explicit Specification”
 - ▶ **Graphical notations**
 - ★ Semantic networks
 - ★ Topic Maps (see <http://www.topicmaps.org/>)
 - ★ UML
 - ★ RDF
 - ▶ **Logic based**
 - ★ Description Logics (e.g., OIL, DAML+OIL, OWL)
 - ★ Rules (e.g., RuleML, LP/Prolog)
 - ★ First Order Logic (e.g., KIF)
 - ★ Conceptual graphs
 - ★ Non-classical logics (e.g., Flogic, Non-Mon, modalities)
 - ▶ **Probabilistic/fuzzy**
- Degree of formality varies widely
 - ▶ Increased formality makes languages more amenable to machine processing (e.g., automated reasoning)

Many languages use “object oriented” model based on:

- **Objects**/Instances/Individuals
 - ▶ Elements of the domain of discourse
 - ▶ Equivalent to constants in FOL
- **Types**/Classes/Concepts
 - ▶ Sets of objects sharing certain characteristics
 - ▶ Equivalent to unary predicates in FOL
- **Relations**/Properties/Roles
 - ▶ Sets of pairs (tuples) of objects
 - ▶ Equivalent to binary predicates in FOL
- Such languages are/can be:
 - ▶ Well understood
 - ▶ Formally specified
 - ▶ (Relatively) easy to use
 - ▶ Amenable to machine processing

Web “Schema” Languages

- Existing Web languages extended to facilitate content description
 - ▶ **XML** → XML Schema (**XMLS**)
 - ▶ **RDF** → RDF Schema (**RDFS**)
- XMLS not an ontology language
 - ▶ Changes format of DTDs (document schemas) to be XML
 - ▶ Adds an extensible type hierarchy
 - ★ Integers, Strings, etc.
 - ★ Can define sub-types, e.g., positive integers
- RDFS is recognisable as an ontology language
 - ▶ **Classes** and **properties**
 - ▶ **Sub/super-classes** (and properties)
 - ▶ **Range** and **domain** (of properties)

RDF and RDFS

- RDF stands for Resource Description Framework
- RDF is a data model for objects and relations between them
- RDF Schema is a vocabulary description language
- Describes properties and classes of RDF resources
 - ▶ Class, Property
 - ▶ type, subClassOf, subPropertyOf
 - ▶ range, domain
- Provides semantics for generalization hierarchies of properties and classes
- RDF is graphical formalism (+ XML syntax + semantics)
 - ▶ for representing metadata
 - ▶ for describing the semantics of information in a machine-accessible way

(<http://www.w3.org/RDF>)

The RDF Data Model

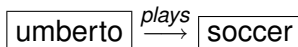
- Statements are of the form

⟨subject, predicate, object⟩

called triples: e.g.

⟨umberto, plays, soccer⟩

- can be represented graphically as:



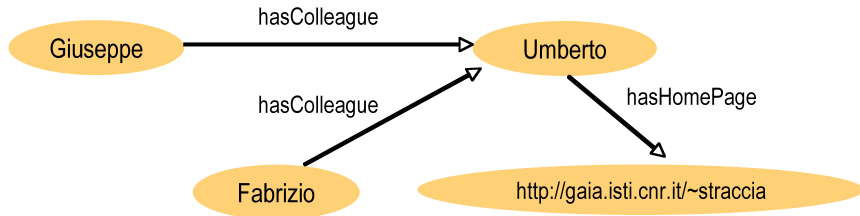
- Statements describe properties of resources
- A resource is any object that can be pointed to by a URI:
 - ▶ a document, a picture, a paragraph on the Web;
 - ▶ `http://www.cs.man.ac.uk/index.html`
 - ▶ a book in the library, a real person (?)
 - ▶ `isbn://5031-4444-3333`
 - ▶ ...
 - ▶ Properties themselves are also resources (URIs)

URIs

- URI = Uniform Resource Identifier
- The generic set of all names/addresses that are short strings that refer to resources
- URLs (Uniform Resource Locators) are a particular type of URI, used for resources that can be accessed on the WWW (e.g., web pages)
- In RDF, URIs typically look like “normal” URLs, often with fragment identifiers to point at specific parts of a document:
- `http://www.somedomain.com/some/path/to/file#fragmentID`

Linking Statements

- The subject of one statement can be the object of another
- Such collections of statements form a directed, labeled graph



- Note that the object of a triple can also be a “literal” (a string)

RDF syntax

- RDF has an XML syntax that has a specific meaning:
- Every Description element describes a resource
- Every attribute or nested element inside a Description is a property of that Resource
- We can refer to resources by using URIs

```
<Description about="some.uri/person/giuseppe_amato">
  <hasColleague resource="some.uri/person/giuseppe_amato"/>
</Description>
<Description about="some.uri/person/umberto_straccia">
  <hasHomePage> http://gaia.isti.cnr.it/~straccia</hasHomePage>
</Description>
<Description about="some.uri/person/fabrizio_sebastiani">
  <hasColleague resource="some.uri/umberto_straccia"/>
</Description>
```



RDF Schema (RDFS)

- RDF gives a formalism for meta data annotation, and a way to write it down in XML, but it does not give any special meaning to vocabulary such as **subClassOf** or **type**
 - ▶ Interpretation is an arbitrary binary relation
- RDF Schema allows you to define vocabulary terms and the relations between those terms
 - ▶ it gives “extra meaning” to particular RDF predicates and resources
 - ▶ this “extra meaning”, or semantics, specifies how a term should be interpreted

RDFS Example

- RDF Schema terms (just a few examples):
 - ▶ Class
 - ▶ Property
 - ▶ type
 - ▶ subClassOf
 - ▶ range
 - ▶ domain
- These terms are the RDF Schema building blocks (constructors) used to create vocabularies:

```
<Person, type, Class>  
<hasColleague, type, Property>  
<Professor, subClassOf, Person>  
<Carole, type, Professor>  
<hasColleague, range, Person>  
<hasColleague, domain, Person>
```

RDF/RDFS “Liberality”

- No distinction between classes and instances (individuals)

```
<Species,type,Class>  
<Lion,type,Species>  
<Leo,type,Lion>
```

- Properties can themselves have properties

```
<hasDaughter,subPropertyOf,hasChild>  
<hasDaughter,type,familyProperty>
```

- No distinction between language constructors and ontology vocabulary, so constructors can be applied to themselves/each other

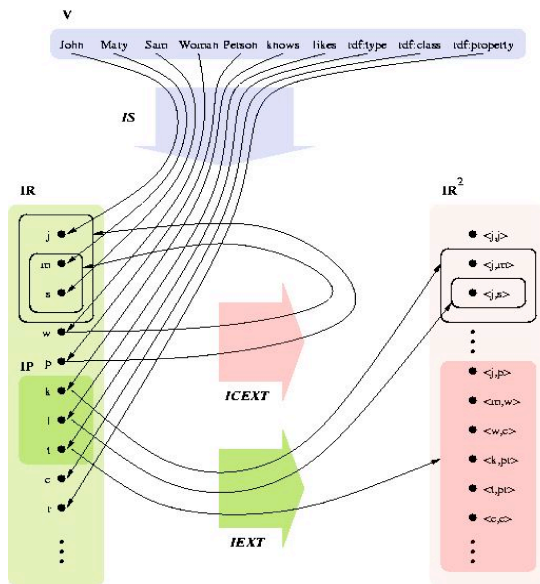
```
<type,range,Class>  
<Property,type,Class>  
<type,subPropertyOf,subClassOf>
```

RDF/RDFS Semantics

- RDF has “Non-standard” semantics in order to deal with this
- Semantics given by RDF Model Theory (MT)
- In RDF MT, an interpretation I of a vocabulary V consists of:
 - ▶ IR , a non-empty set of resources, called the domain of I .
 - ▶ IS , a mapping from URI references in V into IR
 - ▶ IP , a distinguished subset of IR (the set of properties of I)
 - ★ A vocabulary element $v \in V$ is a property iff $IS(v) \in IP$
 - ▶ $IEXT$, a mapping from IP into the powerset of $IR \times IR$, $IEXT(x)$ is called the extension of x
 - ★ I.e., a set of elements $\langle x, y \rangle$, with x, y elements of IR
 - ★ I.e., is a set of pairs which identify the arguments for which the property is true
 - ★ This trick of distinguishing a relation as an object from its relational extension allows a property to occur in its own extension
 - ▶ IL , a mapping from typed literals in V into IR
 - ▶ A distinguished subset LV of IR , called the set of literal values, which contains all the plain literals in V
- Class interpretation $ICEXT$ simply induced by $IEXT(IS(type))$
 - ▶ $ICEXT(C) = \{x \mid \langle x, C \rangle \in IEXT(IS(type))\}$

(<http://www.w3.org/TR/rdf-mt/>)

Example RDF/RDFS Interpretation



RDFS Interpretations

- RDFS adds extra constraints on interpretations
 - ▶ E.g., interpretationss of $\langle C, \text{subClassOf}, D \rangle$ constrained to those where $ICEXT(IS(C)) \subseteq ICEXT(IS(D))$
- Can deal with triples such as



```
<Species,type,Class>  
<Lion,type,Species>  
<Leo,type,Lion>
```

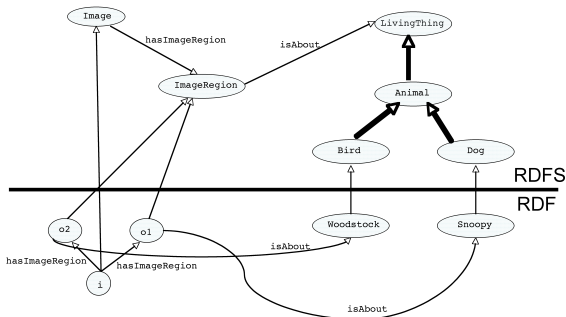
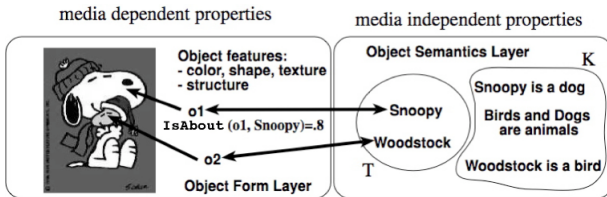


```
<SelfInst,type,SelfInst>
```

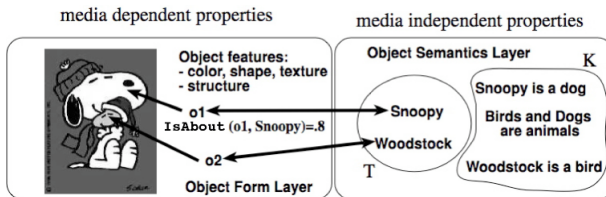
- And even with triples such as
 - ▶ $\langle \text{type}, \text{subPropertyOf}, \text{subClassOf} \rangle$
- But not clear if meaning matches intuition (if there is one)



Example: Multimedia Information retrieval



Example: Multimedia Information retrieval

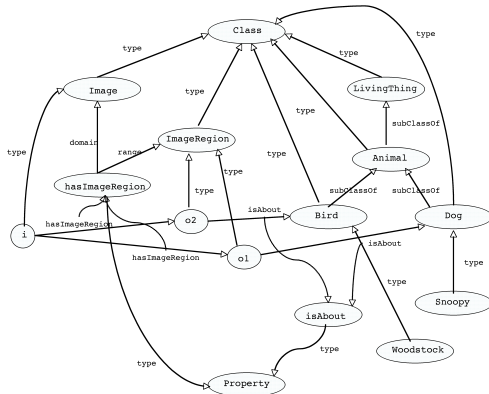
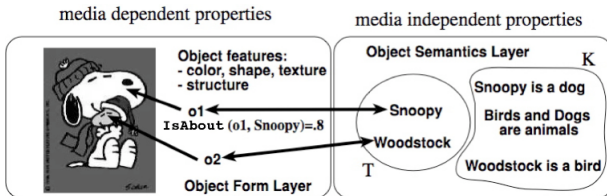


```
<LivingThing,type,Class>
<Image,type,Class>
<ImageRegion,type,Class>
<Animal,type,Class>
<Dog,type,Class>
<Bird,type,Class>
<Animal,subClassOf,LivingThing >
<Dog,subClassOf,Animal>
<Bird,subClassOf,Animal>
<hasImageRegion,type,Property>
<hasImageRegion,domain,Image>
```

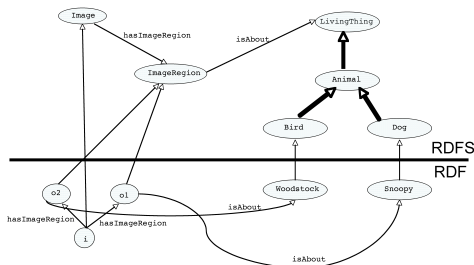
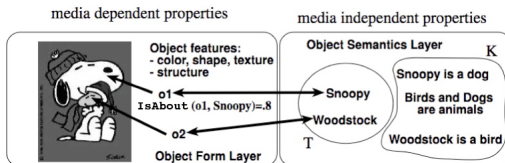
```
<hasImageRegion,range,ImageRegion>
<isAbout,type,Property>
<isAbout,domain,ImageRegion>
<isAbout,range,LivingThing>
<i,type,Image>
<o1,type,ImageRegion>
<o2,type,ImageRegion>
<i,hasImageRegion,o1>
<i,hasImageRegion,o2>
<woodstock,type,Bird>
<snoopy,type,Dog>
<o1,isAbout,snoopy>
<o2,isAbout,woodstock>
```

```
<hasColor,type,Property>
<hasColor,domain,ImageRegion>
<hasColor,range,Color>
<hasShape,type,Property>
<hasShape,domain,ImageRegion>
<hasShape,range,Shape>
<hasTexture,type,Property>
<hasTexture,domain,ImageRegion>
<hasTexture,range,Texture>
```

Example: Multimedia Information retrieval



Example: Multimedia Information retrieval



Query: "Find image regions about animals"

In SPARQL: `SELECT ?r`
`WHERE { ?i isAbout ?o .`
`?o type Animal }`

<http://www.w3.org/TR/rdf-sparql-query/>

Problems with RDFS

- RDFS **too weak** to describe resources in sufficient detail
 - ▶ No **localised range and domain** constraints
 - ★ Can't say that the range of hasChild is person when applied to persons and elephant when applied to elephants
 - ▶ No **existence/cardinality** constraints
 - ★ Can't say that all instances of person have a mother that is also a person, or that persons have exactly 2 parents
 - ▶ No **transitive, inverse or symmetrical** properties
 - ★ Can't say that isPartOf is a transitive property, that hasPart is the inverse of isPartOf or that touches is symmetrical
 - ▶ ...
- Difficult to provide **reasoning support**
 - ▶ No “native” reasoners for non-standard semantics
 - ▶ May be possible to reason via FO axiomatisation
 - ▶ Some reasoners exists: e.g., Jena

<http://jena.sourceforge.net/>

Web Ontology Language Requirements

Desirable features identified for Web Ontology Language:

- Extends existing Web standards
 - ▶ Such as XML, RDF, RDFS
- Easy to understand and use
 - ▶ Should be based on familiar KR idioms
- Formally specified
- Of “adequate” expressive power
- Possible to provide automated reasoning support

From RDF to OWL

Desirable features identified for Web Ontology Language:

- Two languages developed to satisfy above requirements
 - ▶ **OIL**: developed by group of (largely) European researchers (several from EU OntoKnowledge project)
 - ▶ **DAML-ONT**: developed by group of (largely) US researchers (in DARPA DAML programme)
- Efforts merged to produce **DAML+OIL**
 - ▶ Development was carried out by “Joint EU/US Committee on Agent Markup Languages”
 - ▶ Extends (“DL subset” of) RDF
- DAML+OIL submitted to W3C as basis for standardisation
 - ▶ Web-Ontology (**WebOnt**) Working Group formed
 - ▶ WebOnt group developed **OWL** language based on DAML+OIL

<http://www.w3.org/2004/OWL/>

<http://www.schemaweb.info/>